

Wydział: EAIIB	Kierunek: Automatyka i Robotyka	Przedmiot: Wprowadzenie do baz danych	Skład zespołu: Otylia Rybak Kamila Rębilas Michał Rogowski
Data oddania: Czerwiec 2026	Temat: Baza danych i dashboard pogodowy na podstawie Open-Meteo API		Raport końcowy

1. Cel projektu

Celem projektu było zaprojektowanie i zaimplementowanie systemu do automatycznego pobierania, przechowywania oraz wizualizacji danych pogodowych i jakości powietrza. Dane pozyskiwano z darmowego Open-Meteo API dla dziewięciu wybranych lokalizacji w Polsce i Europie. Wyniki importu zapisywano w relacyjnej bazie danych PostgreSQL, a do ich prezentacji opracowano interaktywny dashboard w technologii Streamlit.

System realizuje trzy główne funkcje: cykliczne pobieranie prognoz pogodowych i wskaźników jakości powietrza, ich trwałe przechowywanie z eliminacją duplikatów, oraz wizualizację z możliwością filtrowania według wielu kryteriów. Projekt podzielono na trzy części realizowane przez członków zespołu: warstwę importu danych, schemat bazy danych z zapytaniami SQL oraz dashboard wraz z dokumentacją i raportem końcowym.

2. Wybór technologii

Przy doborze technologii kierowano się kryteriami dostępności, łatwości integracji z bazą danych oraz możliwościami wizualizacyjnymi.

Python 3.13 wybrano jako język implementacji ze względu na bogate ekosystemowe wsparcie dla integracji z bazami danych (biblioteka SQLAlchemy), przetwarzania danych tabelarycznych (pandas) oraz szybkiego tworzenia aplikacji webowych. Do zarządzania zależnościami zastosowano menedżer `uv`, który zapewnia szybką instalację pakietów i deterministyczne środowisko wirtualne poprzez plik `uv.lock`.

PostgreSQL 18 wybrano jako system zarządzania bazą danych ze względu na pełne wsparcie relacyjnego modelu danych z kluczami obcymi, ograniczeniami `UNIQUE` oraz klauzulą `ON CONFLICT`, która umożliwia idempotentny zapis danych bez duplikatów. Alternatywą był SQLite, jednak brak obsługi równoległych połączeń i ograniczone możliwości typów danych skłoniły do wyboru PostgreSQL.

Streamlit wybrano do budowy dashboardu ze względu na możliwość tworzenia interaktywnych aplikacji webowych wyłącznie w Pythonie, bez konieczności pisania kodu HTML, CSS ani JavaScript. Biblioteka oferuje gotowe komponenty takie jak suwaki, selectboxy, wykresy i kafelki metryczne, które bezpośrednio mapują się na potrzeby projektu. W porównaniu do alternatyw takich jak Dash (Plotly) czy Grafana, Streamlit wymaga znacznie

mniej konfiguracji przy podobnych możliwościach prezentacyjnych dla prototypu akademickiego.

Open-Meteo API wybrano ze względu na brak wymagań rejestracyjnych i brak limitu zapytań dla użytku niekomercyjnego, co eliminuje konieczność zarządzania kluczami API. Dane pogodowe opierają się na modelach meteorologicznych ECMWF i DWD o wysokiej dokładności.

3. Opis API

W projekcie wykorzystano trzy endpointy udostępniane przez Open-Meteo:

Endpoint	Opis
<code>api.open-meteo.com/v1/forecast</code>	Dane pogodowe godzinowe i dzienne: temperatura, wilgotność, opady, prędkość wiatru, kod pogody
<code>air-quality-api.open-meteo.com/v1/air-quality</code>	Dane jakości powietrza: PM2.5, PM10, europejski indeks AQI
<code>geocoding-api.open-meteo.com/v1/search</code>	Geokodowanie – pobieranie współrzędnych geograficznych na podstawie nazwy miasta

Tabela 1: Tabela 1. Wykorzystane endpointy Open-Meteo API

API nie wymaga klucza dostępu ani rejestracji. Dane zwracane są w formacie JSON i obejmują prognozy na okres 3 dni z rozdzielczością godzinową. Kody pogodowe zgodne są ze standardem WMO (World Meteorological Organization), co umożliwiło stworzenie tabeli słownikowej z opisami słownymi stanów atmosferycznych.

Monitorowanymi lokalizacjami są: Kraków, Warszawa, Gdańsk, Wrocław, Zakopane, Berlin, Paryż, Londyn oraz Rzym. Dobór lokalizacji obejmuje zarówno duże miasta polskie, jak i europejskie stolicy, co pozwala na porównanie warunków pogodowych w różnych strefach klimatycznych.

4. Model bazy danych

Bazę danych zaprojektowano w PostgreSQL 18. Schemat składa się z czterech tabel merytorycznych, jednej tabeli słownikowej oraz jednej tabeli technicznej do rejestrowania historii importów.

Tabela	Opis
locations	Lokalizacje monitorowane przez system (miasto, szerokość i długość geograficzna)
weather_hourly	Godzinowe dane pogodowe: temperatura, wilgotność, opady, prędkość wiatru, kod pogody
weather_daily	Dzienne prognozy: temperatura min/max, wschód i zachód słońca
air_quality	Godzinowe dane jakości powietrza: PM10, PM2.5, AQI
dict_weather_code	Słownik kodów WMO z opisami słownymi stanów pogody
import_log	Logi importów: czas, status, liczba rekordów, komunikat błędu

Tabela 2: Tabela 2. Tabele w bazie danych

Tabela `locations` pełni rolę centralną – wszystkie tabele pomiarowe odwołują się do niej kluczem obcym `location_id`. Taka normalizacja eliminuje redundancję nazw miast i współrzędnych geograficznych. Tabela `dict_weather_code` przechowuje 28 opisów kodów WMO (np. kod 0 to „Czyste niebo”, kod 95 to „Burza”) i jest powiązana z `weather_hourly` przez `weather_code`. Dzięki temu dashboard może wyświetlać opisy słowne zamiast liczb, co widać na poniższym zrzucie:

 Dane godzinowe

	measurement_time	temperature	humidity	rain	wind_speed	weather_code
0	2026-06-16 00:00:00	12.9	63	0	12.7	Czyste niebo
1	2026-06-16 01:00:00	12.1	66	0	9.3	Przeważnie czyste niebo
2	2026-06-16 02:00:00	11.5	69	0	9.7	Czyste niebo
3	2026-06-16 03:00:00	11	71	0	9.1	Częściowe zachmurzenie
4	2026-06-16 04:00:00	10.5	73	0	8.3	Przeważnie czyste niebo
5	2026-06-16 05:00:00	10.1	74	0	7.6	Przeważnie czyste niebo
6	2026-06-16 06:00:00	10.3	76	0	8	Zachmurzenie całkowite
7	2026-06-16 07:00:00	11.1	75	0	7.9	Częściowe zachmurzenie
8	2026-06-16 08:00:00	12.5	66	0	9.7	Przeważnie czyste niebo
9	2026-06-16 09:00:00	13.7	60	0	11.9	Zachmurzenie całkowite

Rysunek 1: Rysunek 1. Dane godzinowe z opisami kodów pogody

5. Diagram ERD



Rysunek 2: Rysunek 2. Diagram ERD bazy danych

Tabele `weather_hourly`, `weather_daily` oraz `air_quality` powiązane są kluczem obcym z tabelą `locations` (pole `location_id`). Tabela `weather_hourly` powiązana jest dodatkowo z tabelą słownikową `dict_weather_code` przez pole `weather_code`. Każda z tabel pomiarowych posiada ograniczenie `UNIQUE` na kombinacji `location_id` i czasu pomiaru, co zabezpiecza przed duplikatami przy wielokrotnym uruchamianiu importu.

6. Opis importu danych

Import danych zaimplementowano w pliku `api_import.py` z użyciem standardowej biblioteki `urllib` do komunikacji z API oraz `SQLAlchemy` do zapisu w bazie danych. Połączenie z PostgreSQL realizowane jest przez sterownik `psycopg2`. Proces importu przebiega w następujących etapach:

1. Geokodowanie nazw miast z pliku konfiguracyjnego przy użyciu Geocoding API.
2. Zapis lub aktualizacja współrzędnych lokalizacji w tabeli `locations`.
3. Dla każdej lokalizacji: pobranie danych pogodowych (godzinowych i dziennych) oraz danych jakości powietrza.
4. Zapis rekordów do odpowiednich tabel z użyciem klauzuli `ON CONFLICT DO UPDATE`, co eliminuje duplikaty.
5. Zapis wpisu do tabeli `import_log` ze statusem `SUCCESS` lub `FAILED` i liczbą dodanych rekordów.

Import uruchamiany jest cyklicznie co 60 minut przez wbudowany scheduler. Możliwe jest również jednorazowe uruchomienie przez parametr `--once` lub z niestandardowym inter-

wałem przez `--interval`. Przy pierwszym uruchomieniu skrypt automatycznie tworzy bazę danych i wszystkie tabele, co eliminuje konieczność ręcznej konfiguracji schematu.

7. Skrypty SQL

Przygotowano pliki `create_tables.sql` oraz `insert_initial_data.sql` umożliwiające odtworzenie bazy danych od zera bez uruchamiania skryptu Pythona. Poniżej przedstawiono przykładową definicję tabeli `weather_hourly` ilustrującą zastosowanie kluczy obcych i ograniczenia unikalności:

```
CREATE TABLE IF NOT EXISTS weather_hourly (
    id SERIAL PRIMARY KEY,
    location_id INT NOT NULL,
    measurement_time TIMESTAMP NOT NULL,
    temperature NUMERIC(4, 1),
    humidity INT,
    rain NUMERIC(4, 1),
    wind_speed NUMERIC(4, 1),
    weather_code INT,
    CONSTRAINT fk_location
        FOREIGN KEY (location_id) REFERENCES locations(id),
    CONSTRAINT fk_weather_code
        FOREIGN KEY (weather_code)
            REFERENCES dict_weather_code(weather_code),
    CONSTRAINT unique_location_time
        UNIQUE (location_id, measurement_time)
);
```

8. Zapytania analityczne SQL

Przygotowano pięć zapytań analitycznych zaimplementowanych w pliku `queries.py`. Zapytania umożliwiają analizę danych bez użycia dashboardu – mogą być uruchamiane bezpośrednio z linii poleceń lub z poziomu narzędzia pgAdmin.

Zapytanie 1 – Aktualny przegląd pogody (ostatnie 24h):

```
SELECT l.city_name AS miasto,
       w.measurement_time AS data_godzina,
       w.temperature AS temperatura_c,
       w.humidity AS wilgotnosc_procent,
       w.wind_speed AS wiatr_kmh,
       d.description AS stan_pogody
FROM weather_hourly w
JOIN locations l ON w.location_id = l.id
JOIN dict_weather_code d ON w.weather_code = d.weather_code
ORDER BY w.measurement_time DESC
LIMIT 24;
```

Zapytanie 2 – Prognoza dzienna:

```

SELECT l.city_name AS miasto,
       wd.forecast_date AS data_proгноzy,
       wd.temp_max AS maksymalna_temperatura,
       wd.temp_min AS minimalna_temperatura,
       wd.sunrise AS wschod_slonca,
       wd.sunset AS zachod_slonca
FROM weather_daily wd
JOIN locations l ON wd.location_id = l.id
ORDER BY wd.forecast_date ASC;

```

Zapytanie 3 – Alerty i anomalie pogodowe:

Zapytanie wykrywa potencjalnie niebezpieczne warunki atmosferyczne: silny wiatr (powyżej 25 km/h), temperaturę poniżej zera oraz kody pogodowe odpowiadające intensywnym opadom deszczu (65, 82) i burzom (95, 99).

```

SELECT l.city_name AS miasto,
       w.measurement_time AS czas_zdarzenia,
       w.temperature AS temperatura,
       w.wind_speed AS predkosc_wiatru,
       d.description AS zjawisko
FROM weather_hourly w
JOIN locations l ON w.location_id = l.id
JOIN dict_weather_code d ON w.weather_code = d.weather_code
WHERE w.wind_speed > 25.0
      OR w.temperature < 0.0
      OR w.weather_code IN (65, 82, 95, 99)
ORDER BY w.measurement_time DESC;

```

Zapytanie 4 – Średnie dobowe zanieczyszczenie powietrza:

```

SELECT l.city_name AS miasto,
       DATE(a.measurement_time) AS dzien,
       ROUND(AVG(a.pm10), 1) AS srednie_pm10,
       ROUND(AVG(a.pm2_5), 1) AS srednie_pm25,
       MAX(a.aqi) AS najwyzsze_aqi
FROM air_quality a
JOIN locations l ON a.location_id = l.id
GROUP BY l.city_name, DATE(a.measurement_time)
ORDER BY dzien DESC;

```

Zapytanie 5 – Statystyki techniczne systemu:

```

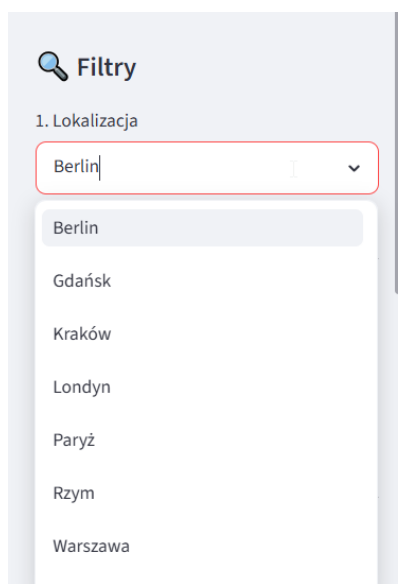
SELECT status,
       COUNT(*) AS liczba_uruchomien,
       SUM(records_added) AS lacznie_pobranych_rekordow
FROM import_log
GROUP BY status;

```

9. Opis dashboardu

Dashboard zaimplementowano przy użyciu biblioteki Streamlit w pliku `dashboard/dashboard.py`. Połączenie z bazą danych nawiązywane jest przez SQLAlchemy z użyciem dekoratora `@st.cache_resource`, który zapewnia utrzymanie jednego połączenia przez całą sesję użytkownika.

Panel boczny zawiera 12 filtrów pogrupowanych tematycznie: wybór lokalizacji i typu danych (godzinowe/dziennie/jakość powietrza), zakres dat i godzin, progi temperaturowe z opcją wyświetlenia linii referencyjnych na wykresie, maksymalne wartości opadu i wiatru, filtr kodu pogody oraz progi PM2.5, PM10 i AQI. Na górze strony wyświetlane są kafelki podsumowujące aktualne wartości.



Rysunek 3: Rysunek 3. Panel filtrów dashboardu



Rysunek 4: Rysunek 4. Dashboard – widok ogólny z sekcją alertów pogodowych

Widok godzinowy prezentuje wykresy temperatury (liniowy), opadów (słupkowy) oraz prędkości wiatru (liniowy z linią limitu). Dla wiatru linia limitu jest zawsze widoczna – pozwala to ocenić, czy zmierzone wartości przekraczają ustawiony próg bez ukrywania danych. Dla temperatury linie limitów są opcjonalne i domyślnie wyłączone.



Rysunek 5: Rysunek 5. Wykresy temperatury, opadów i prędkości wiatru

Sekcja jakości powietrza zawiera wykres AQI w czasie oraz wykresy PM2.5 i PM10 wyświetlane obok siebie. Widok dostępny jest zarówno w zakładce danych godzinowych, jak i jako osobna zakładka „Jakość powietrza”.



Rysunek 6: Rysunek 6. Wykresy jakości powietrza (AQI, PM2.5, PM10)

10. Wnioski

W ramach projektu zrealizowano kompletny system do pobierania, przechowywania i wizualizacji danych pogodowych. Zastosowanie Open-Meteo API umożliwiło bezpłatny dostęp do danych pogodowych i jakości powietrza w czasie rzeczywistym, bez konieczności rejestracji ani zarządzania kluczami dostępu.

Relacyjna baza danych PostgreSQL z normalizacją przez tabelę lokalizacji i słownik kodów pogodowych zapewniła spójność danych i czytelność wyników. Ograniczenia `UNIQUE` i klauzula `ON CONFLICT` wyeliminowały problem duplikatów przy cyklicznym imporcie.

Biblioteka Streamlit okazała się trafnym wyborem dla prototypu akademickiego – umożliwiła szybkie stworzenie interaktywnego dashboardu z 12 filtrami i wieloma typami wykresów wyłącznie w Pythonie, bez potrzeby znajomości technologii frontendowych. Zastosowanie `@st.cache_resource` dla połączenia z bazą danych i parametryzowanych zapytań SQL wyeliminowało problemy z wydajnością i bezpieczeństwem.

Projekt wykazał, że architektura oparta na automatycznym imporcie danych z API z harmonogramem czasowym, relacyjnej bazie danych i warstwie wizualizacyjnej jest praktycznym i łatwym w utrzymaniu rozwiązaniem dla systemów monitorowania środowiskowego.